
Smart Grocery & Meal Planner: An Autonomous AI-Powered Budget and Nutrition Optimization System

Mrunal Kavade

Dept. of Electronics and Telecommunication Engineering
MKSSS's Cummins College of Engineering for Women
Pune, India
UEC2023324

Anshita Kayande

Dept. of Electronics and Telecommunication Engineering
MKSSS's Cummins College of Engineering for Women
Pune, India
UEC2023325

Utkarsha Parmale

Dept. of Electronics and Telecommunication Engineering
MKSSS's Cummins College of Engineering for Women
Pune, India
UEC2023338

Aarti Patil

Dept. of Electronics and Telecommunication Engineering
MKSSS's Cummins College of Engineering for Women
Pune, India
UEC2023339

Abstract Household food management suffers from persistent inefficiencies including untracked inventory, duplicate purchasing, food wastage from unmonitored expiry, and budget-suboptimal shopping across competing vendors. This paper presents the Smart Grocery & Meal Planner, an autonomous AI-powered web application designed to address these challenges through integrated inventory awareness, multi-vendor price optimization, and context-grounded recipe generation. The system employs a Flask-based RESTful backend, a SQLite persistence layer, and Google Gemini as its generative AI engine, orchestrated via a modular agents architecture. Key innovations include a heuristic expiry prediction engine, a real-time cross-platform cost comparator, and a context-injected AI Chef that operates with complete pantry awareness. Unlike RAG-based systems, the platform employs direct context injection, achieving 100% inventory accuracy for household-scale datasets. Evaluation demonstrates significant reduction in manual grocery management effort, accurate expiry alerting, and high-quality recipe generation aligned to available ingredients.

Keywords — smart grocery, meal planning, pantry management, AI recipe generation, price optimization, expiry prediction, context injection, Flask, SQLite, Google Gemini, prompt engineering, food waste reduction

I. INTRODUCTION

Modern household food management is fragmented across multiple cognitive tasks: remembering what is in the refrigerator, tracking expiry dates, comparing grocery prices across competing platforms, and deciding what meals to prepare with available ingredients. Studies indicate that the average household discards approximately 30% of purchased food due to overlooked expiry and poor pantry visibility.

Existing solutions—static list-making apps and manual spreadsheets—fail to address the dynamic, interconnected nature of grocery management. They require continuous manual updates, provide no price intelligence, and offer no AI-driven culinary guidance. This creates a significant gap between the user's intent to manage groceries efficiently and the tools available to support that intent.

The Smart Grocery & Meal Planner was designed as a holistic autonomous agent to close this gap. Rather than requiring manual entry or passive reminders, the system actively orchestrates inventory state, cost optimization, expiry alerting, and recipe generation within a unified mobile-first interface. The principal contributions of this work are: (i) a heuristic expiry prediction engine requiring no user input; (ii) a multi-store price comparison algorithm that identifies the cheapest shopping basket in $O(N)$ time; (iii) a context-injected AI Chef that generates recipes grounded in real-time pantry state; and (iv) a stateful conversational chatbot that serves as a persistent kitchen manager.

II. RELATED WORK

Household inventory management systems have been explored as components of smart home ecosystems. Early approaches relied on barcode scanning for item addition, which introduced friction and limited adoption. Computer-vision-based systems using YOLO and similar architectures have demonstrated promise for automatic pantry detection but require dedicated hardware.

AI-powered recipe generation has been studied extensively using transformer-based language models. Recurrent approaches modeled ingredient co-occurrence to suggest recipes from partial ingredient lists. Large language models (LLMs) introduced by Brown et al. [1] demonstrated that few-shot prompting enables domain-specific reasoning without fine-tuning, a finding directly applicable to recipe generation from inventory context.

Price optimization in grocery contexts has typically required web scraping or proprietary retailer APIs. The present system adopts a vendor-agnostic multi-store price model that simulates competitive pricing, enabling the optimization engine to function without live API dependencies while preserving algorithmic validity.

The use of context injection as an alternative to Retrieval-Augmented Generation (RAG) has been noted for small, structured datasets where full-context inclusion is feasible within the model's context window [2]. This approach eliminates retrieval latency and guarantees completeness, making it well-suited for household-scale inventories.

III. SYSTEM ARCHITECTURE

A. Overview

The system follows a three-tier architecture: a mobile-first Single Page Application (SPA) frontend, a Python Flask RESTful backend, and a SQLite persistence layer. All AI logic is abstracted into a dedicated `agents.py` module, enforcing separation of concerns and enabling independent testing of each functional component.

The frontend employs a view-switching paradigm—hiding and showing HTML sections—to deliver a native-app-like

experience without framework overhead. CSS custom properties implement a glassmorphism design system with dynamic status badges for expiry visualization.

B. Agent Architecture (`agents.py`)

Four primary agent functions are defined in the logic layer, each responsible for a distinct operational domain:

- **Receipt Parser Agent:** Receives OCR text and returns structured JSON containing item names, quantities, and inferred categories using few-shot prompting.
- **Recipe Generator Agent:** Accepts a serialized pantry snapshot and returns recipe suggestions with ingredient lists and preparation steps. Missing ingredients are flagged with a structured JSON identifier for one-click cart integration.
- **Nutrition Estimator Agent:** Queries the OpenFoodFacts API for macronutrient data, falling back to category-based heuristics when the item is unavailable or locally sourced.
- **Chatbot Agent:** A stateful conversational agent that receives the full pantry context on every request, ensuring responses remain grounded in current inventory reality.

C. Data Flow

The complete data flow for a recipe query proceeds as follows:

1. User submits query via ChatView in the SPA.
2. Backend fetches all `PantryItem` records from SQLite.
3. Context Assembly constructs a prompt injecting the full pantry list.
4. Gemini AI engine returns a structured JSON recipe object.
5. Backend compares required ingredients against DB; missing items are flagged.
6. Frontend renders a Recipe Card; missing items display an Add to Cart action button.

IV. IMPLEMENTATION

A. Technology Stack

Component	Technology	Rationale
Frontend	HTML5, Vanilla CSS3, ES6 JS	Lightweight SPA; no framework overhead; native DOM control
Backend	Python 3.11, Flask	Minimal footprint; ideal for AI logic routing and REST APIs
Database	SQLite via SQLAlchemy ORM	Serverless, portable, zero-config persistence for single-user scope
AI Engine	Google Gemini (GenAI SDK)	1M-token context window; superior structured JSON output reliability
Images	Unsplash / Pexels API	Watermark-free, high-resolution grocery product imagery
Nutrition	OpenFoodFacts / Spoonacular	Open nutritional database with heuristic fallback for local items

TABLE I. TECHNOLOGY STACK

B. Smart Cart Optimization Algorithm

The cost optimization module operates in $O(N)$ time over the set of `CartItem` records. For each item i in the cart, the system retrieves a `prices_json` object containing price mappings for each vendor store S_j . A cumulative sum is calculated per store:

$$\text{Total}(S_j) = \sum_{\forall \text{ item}_i \in \text{Cart}} \text{prices_json}[\text{item}_i][S_j]$$

The algorithm identifies the global minimum store: $\text{Best_Store} = \text{argmin}_j \text{Total}(S_j)$, and computes potential savings as $\text{Savings} = \text{Total}(S_{\text{max}}) - \text{Total}(S_{\text{min}})$. The result is surfaced in the UI as a highlighted recommendation with a calculated savings badge.

C. Expiry Prediction Engine

Upon pantry transfer, the backend automatically assigns an expiry date using a rule-based categorical engine that requires zero user input:

- Fresh / Vegetable: `current_date + 4 days`
- Dairy products: `current_date + 6 days`
- Frozen items: `current_date + 25 days`
- Packaged goods: `current_date + 60 days`

The UI dynamically evaluates `datetime.utcnow()` against each stored `expiry_date` to render color-coded status

badges: Red (expired), Orange (expiring within 2 days), Green (fresh).

D. Pantry Auto-Update: Transfer Protocol

Upon checkout confirmation, the system executes a database Transfer Protocol. All records from the `cart_item` table are migrated to the `pantry_item` table using an Upsert strategy: if an item name matches an existing pantry record (case-insensitive ilike comparison), the system increments the quantity field rather than inserting a duplicate row. This prevents inventory inflation and maintains data integrity across repeated purchases.

E. Context Injection vs. RAG

The system deliberately employs Context Injection rather than Retrieval-Augmented Generation. In a RAG pipeline, a vector similarity search retrieves relevant document chunks before each AI call, introducing retrieval latency and the risk of incomplete context. Since a household pantry typically contains fewer than 100 items, the entire database state can be serialized into the system prompt within Gemini's context window. This guarantees 100% inventory coverage on every AI request, eliminates vector database infrastructure costs, and removes retrieval failure as a failure mode.

V. PROMPT ENGINEERING

The project employs four distinct prompting strategies to convert Gemini's generative output into deterministic, database-ready JSON structures:

A. Zero-Shot Prompting

Used for atomic classification tasks such as item category assignment. Example: "Classify 'Milk' as one of: Grocery, Dairy, Fresh, or Frozen. Return only the category name." The constrained output space prevents hallucination on well-defined categorical tasks.

B. Few-Shot Prompting (Receipt Parser)

Three annotated examples of receipt text and their target JSON structures are prepended to each parsing request. This approach enables the model to generalize across receipt formats (supermarkets, online order confirmations, handwritten lists) without fine-tuning.

C. Structured Prompting (JSON Mode)

All backend-bound AI responses are requested in strict JSON format with explicit schema specification. The prompt includes a schema definition and a constraint: "Respond ONLY in valid JSON. Do not use markdown code blocks or backticks." The backend wraps all parsing in try-except blocks to handle rare formatting deviations gracefully.

D. Hallucination Control via Contextual Constraints

The chatbot and recipe agents include an explicit constraint in the system prompt: "You are a kitchen assistant. The user's current pantry contains exactly these items: [serialized list]. Do not suggest ingredients or recipes requiring items not in this list unless explicitly asked." This constraint bounds the model's output space to verified inventory, eliminating confabulated ingredient suggestions.

VI. API ECOSYSTEM & SELECTION RATIONALE

The system integrates four external APIs, each selected based on specific functional and quality criteria:

- Unsplash & Pexels (Imagery): Selected for high-resolution, watermark-free grocery photography.

The system constructs precision search queries by appending 'packet product' or 'raw vegetable' with negative keywords (-dish, -cooked, -plating) to ensure product-style imagery rather than prepared food photographs.

- OpenFoodFacts (Nutrition): An open-source nutritional database providing macronutrient data. When an item is unavailable (local or informal products), the system falls back to category-based heuristic estimates validated against standard dietary reference tables.
- Google Gemini (AI Engine): Selected for its 1M-token context window, enabling full pantry injection without truncation, and for its reliable structured JSON output mode essential for database integration.
- Rejected Alternatives: Google Cloud Vision was excluded due to OCR cost and complexity; the Gemini LLM achieves equivalent receipt parsing via structured prompting. Amazon Product API was excluded due to commercial partnership requirements incompatible with academic project scope.

VII. RESULTS AND DISCUSSION

A. Feature Validation

Core system features were validated across 50 test sessions simulating typical household usage patterns. The expiry prediction engine correctly categorized 100% of tested items using the rule-based heuristic, with no manual date entry required. The Transfer Protocol successfully executed pantry updates for all checkout events, with the upsert logic correctly handling duplicate items in 100% of test cases.

B. Cost Optimization Accuracy

The cart optimization algorithm was validated against manually computed store totals for 30 shopping baskets of varying composition. The system correctly identified the minimum-cost store in all 30 cases, with savings calculations accurate to two decimal places. Mean potential savings identified per basket was 12.4% relative to the highest-cost store.

C. AI Recipe Generation Quality

In informal evaluation with 20 users, AI-generated recipes were rated as practically executable with available pantry items in 85% of cases. The missing ingredient detection module correctly identified absent items in 92% of recipe generations, with one-click cart integration functioning reliably for all flagged items.

Metric	Result
Expiry Prediction Accuracy	100% (rule-based, n=50)
Cart Upsert Protocol Success Rate	100% (n=50)
Cost Optimization Correctness	100% (n=30 baskets)
Mean Savings Identified per Basket	12.4% vs. highest-cost store
Recipe Executability Rating	85% user approval (n=20)
Missing Ingredient Detection Accuracy	92% (n=50 recipe generations)
Context Injection Coverage	100% pantry items per AI request
Image Precision (V11 Logic)	94% product-style imagery returned

TABLE II. EVALUATION METRICS SUMMARY

VIII. CHALLENGES AND ENGINEERING SOLUTIONS

A. Cooked-Dish Image Contamination

Initial image search queries returned photographs of prepared dishes (e.g., tomato soup) instead of raw or packaged grocery items. Resolution: V11 Precision Search Logic was implemented, appending category-specific suffixes ('raw vegetable', 'packet product') and prepending negative constraint keywords (-dish, -cooked, -plating, -meal) to all Unsplash and Pexels API queries. This reduced irrelevant image results from approximately 40% to under 6%.

B. Database Schema Evolution

Adding the quantity column to the CartItem model after the database had already been initialized caused schema mismatch errors. Resolution: A dedicated Python migration script (migrate_cart.py) was developed using raw SQL ALTER TABLE commands to update existing database instances without data loss, establishing a migration-first workflow for all subsequent schema changes.

C. Cart Duplication on Repeated Additions

Repeated additions of the same grocery item created duplicate rows rather than incrementing quantity.

Resolution: The CartItem model was refactored to include a quantity field, and the backend route was rewritten to execute a check-before-insert pattern using case-insensitive name matching (ilike), updating quantity on match and inserting only on no-match.

IX. LIMITATIONS

- **Simulated Pricing:** Current vendor prices are statically simulated. Real-time scraping via Selenium or Playwright has not been implemented due to platform terms-of-service constraints.
- **Single-User Scope:** The SQLite backend is optimized for single-user household deployment and does not support concurrent multi-user access or collaborative pantry sharing without migration to a server-grade RDBMS.
- **English Language Only:** All prompt templates and AI interactions are optimized for English-language inputs. Regional language support has not been validated.
- **Manual Receipt Upload:** The receipt parser requires manual file upload; automated purchase history integration with grocery platforms is unavailable without partner API access.

X. FUTURE SCOPE

- Real-Time Price Scraping: Integrating Playwright-based scrapers to fetch live prices from Blinkit, Zepto, and BigBasket, replacing the current simulated pricing model.
- Barcode Integration: Implementing camera-based barcode scanning using ZXing or QuaggaJS to enable effortless pantry population from physical product packaging.
- Predictive Reordering: Applying supervised ML models (Scikit-Learn) trained on historical purchase frequency to predict depletion events and suggest proactive reordering.
- Collaborative Pantry: Extending the data model and authentication layer to support multi-user household accounts with real-time shared inventory state.
- Health Platform Integration: Connecting with Google Fit or Apple Health APIs to calibrate meal recommendations against individual daily caloric expenditure data.

XI. CONCLUSION

This paper presented the Smart Grocery & Meal Planner, an autonomous AI-powered system that addresses the interconnected challenges of household inventory management, multi-vendor price optimization, expiry tracking, and context-aware recipe generation. By integrating a heuristic expiry engine, a cross-platform cost comparator, and a context-injected AI Chef within a unified mobile-first interface, the system delivers meaningful automation across the full lifecycle of household food consumption.

The deliberate choice of context injection over RAG architecture demonstrates that for household-scale datasets, completeness and latency advantages of full-context inclusion outweigh the scalability benefits of vector retrieval. Empirical evaluation confirms high accuracy across all functional modules, with the optimization engine achieving consistent identification of the minimum-cost shopping basket and the AI recipe module delivering executable suggestions grounded in real pantry inventory.

The path forward points toward real-time vendor integration, barcode-based data entry, and predictive restocking capabilities. This work establishes that thoughtful orchestration of accessible LLM infrastructure, combined with disciplined prompt engineering, can produce genuinely useful tools for everyday household intelligence.

ACKNOWLEDGMENT

The authors thank their faculty mentors and project supervisors for their guidance and constructive feedback throughout the development and evaluation of this system prototype.

REFERENCES

- [1] T. Brown et al., "Language models are few-shot learners," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
- [3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [4] A. Kumar, S. Rao, and P. Gupta, "Semantic recipe-ingredient matching using transformer embeddings," in *Proc. IEEE Int. Conf. NLP*, 2023, pp. 312–319.
- [5] OpenFoodFacts, "Open Food Facts database," [Online]. Available: <https://world.openfoodfacts.org/>, 2024.
- [6] Google DeepMind, "Gemini technical report," [Online]. Available: <https://deepmind.google/technologies/gemini/>, 2024.
- [7] Unsplash Inc., "Unsplash API documentation," [Online]. Available: <https://unsplash.com/developers>, 2024.
- [8] SQLAlchemy, "SQLAlchemy documentation," [Online]. Available: <https://www.sqlalchemy.org/>, 2024.
- [9] Flask, "Flask documentation," [Online]. Available: <https://flask.palletsprojects.com/>, 2024.
- [10] M. Woebot Health, "Efficacy of AI-driven context-aware coaching," *J. Medical Internet Research*, vol. 24, e38248, 2022.